

ANTIGRAVITY ENTERPRISE PROMPT

AI Network Config Diff Reviewer — Android APK + Backend + Docker

INSTRUCTIONS FOR USE: Paste this entire prompt into Antigravity to generate the full production-grade source code. After generation, import the android/ module into Android Studio to build the APK, and run docker-compose up --build in the backend/ module to spin up all services.

MASTER ENTERPRISE PROMPT — FULL SOURCE CODE GENERATION

Generate a complete, production-grade, enterprise-level Android APK application with a fully featured backend, database, AI integration, and Docker containerization for the AI Network Config Diff Reviewer platform. Every file must be production-ready, fully implemented with no stubs, no placeholder TODO comments, and no incomplete logic.

SECTION 1 — PROJECT IDENTITY & CONTEXT

Application Name: AI Network Config Diff Reviewer Package Name: com.enterprise.netconfigdiff
Version: 1.0.0 Min SDK: API 26(Android 8.0 Oreo) Target SDK: API 35 (Android 15) Build Tools: Gradle 8.5+ with Kotlin DSL (build.gradle.kts)
Language: Kotlin 2.0+ UI Framework: Jetpack Compose (Material Design 3)

Business Purpose: An enterprise mobile application that enables network engineers and IT managers to upload two versions of network configuration files (AWS Security Groups, GCP Firewall Rules, Azure NSGs, Terraform IaC, JSON/YAML configs), automatically diff them using DeepDiff, send changes to a local Llama 3 model via Ollama for intelligent AI risk analysis, score risks as Low / Medium / High / Critical, validate against compliance frameworks (CIS, NIST, PCI-DSS), and generate PDF/Markdown approval reports — all with full enterprise approval workflows, role-based access control, audit logging, and PostgreSQL persistence, with the entire backend containerized in Docker.

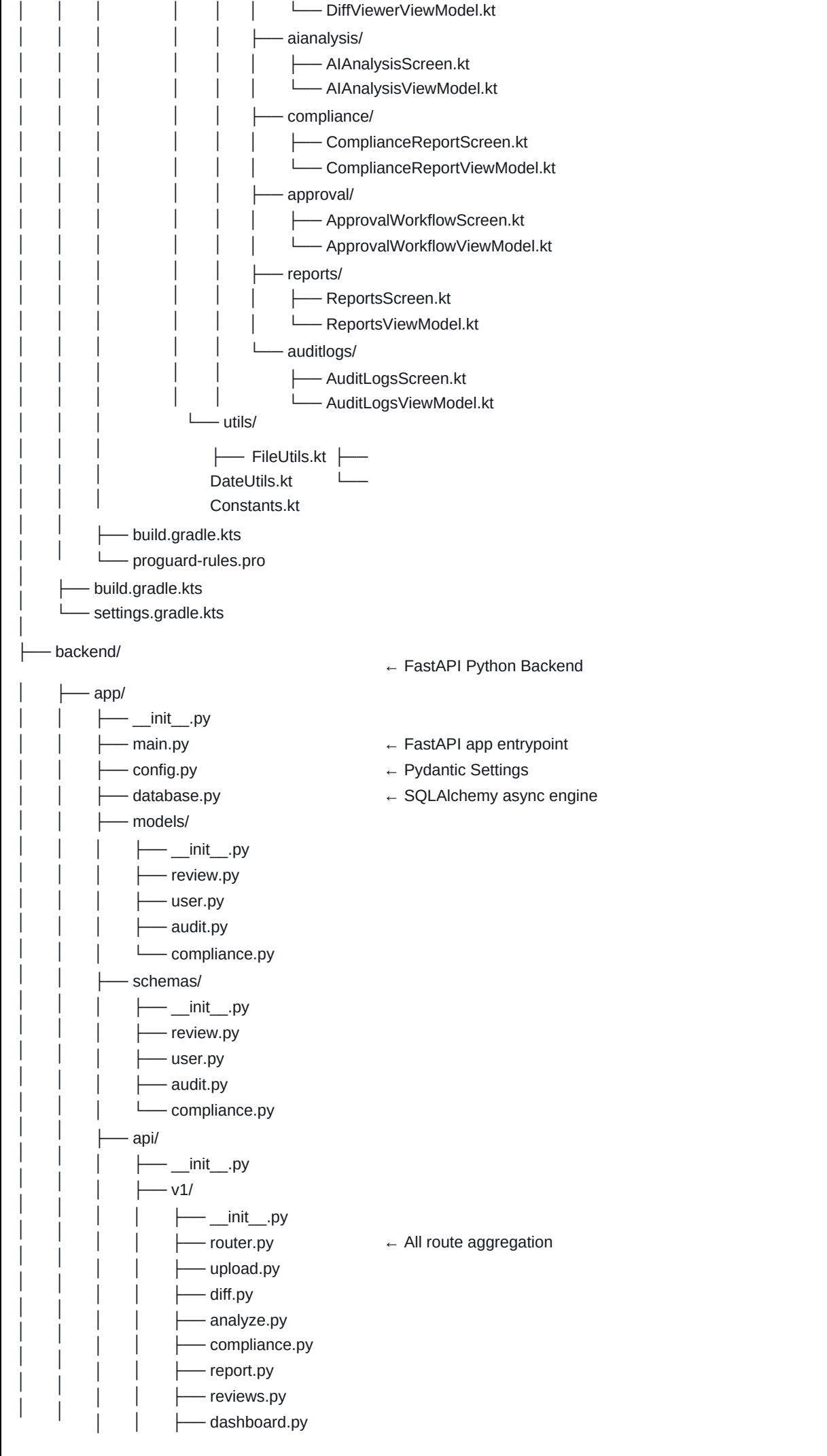
SECTION 2 — PROJECT STRUCTURE (COMPLETE FILE TREE)

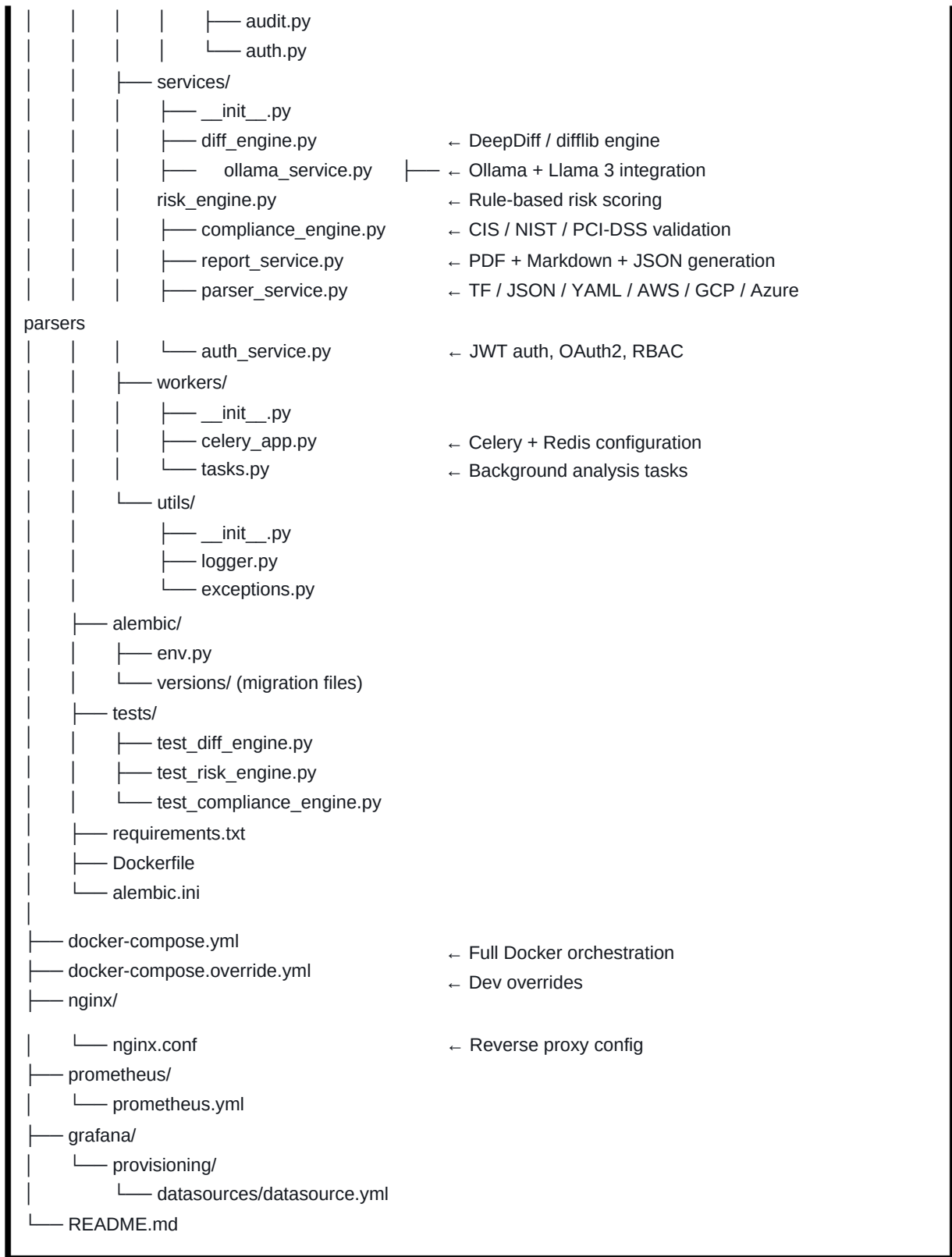
Generate the following directory structure completely:

```

project-root/ └──
android/                                             ← Android Studio Project (APK source)
├──
├── app/
├──   └── src/main/
├──     ├── AndroidManifest.xml
├──     └── java/com/enterprise/netconfigdiff/
├──       ├── MainActivity.kt
├──       ├── NetConfigApp.kt                               ← Application class (Hilt)
├──       ├── di/
├──         ├── AppModule.kt
├──         ├── NetworkModule.kt
├──         └── DatabaseModule.kt
├──       ├── data/
├──         ├── remote/
├──           ├── api/NetConfigApiService.kt
├──           ├── dto/ (all request/response DTOs)
├──           └── interceptors/AuthInterceptor.kt
├──         ├── local/
├──           ├── db/AppDatabase.kt
├──           ├── dao/ (all DAOs)
├──           ├── entity/ (all Room entities)
├──           └── repository/ (all Repository implementations)
├──       ├── domain/
├──         ├── model/ (all domain models)
├──         ├── repository/ (all Repository interfaces)
├──         └── usecase/ (all Use Cases)
├──       ├── presentation/
├──         ├── navigation/
├──           ├── AppNavGraph.kt
├──           └── Screen.kt
├──         ├── theme/
├──           ├── Color.kt
├──           ├── Theme.kt
├──           └── Typography.kt
├──         ├── components/
├──           ├── RiskBadge.kt
├──           ├── DiffLineltem.kt
├──           ├── ComplianceFindingCard.kt
├──           ├── ApprovalStatusChip.kt
├──           ├── LoadingOverlay.kt
├──           └── ErrorSnackBar.kt
├──         ├── screens/
├──           ├── splash/SplashScreen.kt
├──           ├── login/
├──             ├── LoginScreen.kt
├──             └── LoginViewModel.kt
├──           ├── dashboard/
├──             ├── DashboardScreen.kt
├──             └── DashboardViewModel.kt
├──           ├── upload/
├──             ├── UploadConfigScreen.kt
├──             └── UploadConfigViewModel.kt
├──           ├── diffviewer/
├──             └── DiffViewerScreen.kt

```





SECTION 3 — ANDROID APP: FULL SPECIFICATION

3.1 — Architecture

Implement strict Clean Architecture layered as:

- Presentation Layer: JetpackCompose screens + ViewModels (MVVM)

- Domain Layer: Use Cases, Repository interfaces, Domain Models (pure Kotlin, zero Android framework dependencies)
- Data Layer: Repository implementations, Retrofit API service, Room local DB, DataStore preferences

3.2 — Dependencies (build.gradle.kts — app level)

kotlin

```
// Kotlin implementation("org.jetbrains.kotlin:kotlinx-coroutines-android:1.8.1")
implementation("org.jetbrains.kotlin:kotlinx-serialization-json:1.7.1")

// Jetpack Compose BOM
val composeBom = platform("androidx.compose:compose-bom:2024.09.00")
implementation(composeBom)
implementation("androidx.compose.ui:ui")
implementation("androidx.compose.ui:ui-tooling-preview")
implementation("androidx.compose.material3:material3")
implementation("androidx.compose.material:material-icons-extended")
implementation("androidx.activity:activity-compose:1.9.2")
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.6")
implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.6")

// Navigation
implementation("androidx.navigation:navigation-compose:2.8.2")

// Hilt (Dependency Injection)
implementation("com.google.dagger:hilt-android:2.52")
kapt("com.google.dagger:hilt-compiler:2.52")
implementation("androidx.hilt:hilt-navigation-compose:1.2.0")

// Retrofit + OkHttp + Gson
implementation("com.squareup.retrofit2:retrofit:2.11.0")
implementation("com.squareup.retrofit2:converter-gson:2.11.0")
implementation("com.squareup.okhttp3:okhttp:4.12.0")
implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")

// Room (Local Database)
implementation("androidx.room:room-runtime:2.6.1")
implementation("androidx.room:room-ktx:2.6.1")
kapt("androidx.room:room-compiler:2.6.1")

// DataStore (Preferences)
implementation("androidx.datastore:datastore-preferences:1.1.1")

// WorkManager (Background file upload)
implementation("androidx.work:work-runtime-ktx:2.9.1")

// Coil (Image / File Preview)
implementation("io.coil-kt:coil-compose:2.7.0")

// PDF Viewer
implementation("io.github.afreakyelf:Pdf-Viewer:2.1.1")

// File Picker
implementation("com.github.skydoves:landscapist-glide:2.3.4")

// Security / JWT decode
implementation("io.jsonwebtoken:jjwt-api:0.12.3")
implementation("io.jsonwebtoken:jjwt-impl:0.12.3")

// Paging 3
implementation("androidx.paging:paging-runtime-ktx:3.3.2")
```

```
implementation("androidx.paging:paging-compose:3.3.2")

// Lottie animations
implementation("com.airbnb.android:lottie-compose:6.5.0")

// Testing
testImplementation("junit:junit:4.13.2")
testImplementation("org.mockito.kotlin:mockito-kotlin:5.4.0")
androidTestImplementation("androidx.test.espresso:espresso-core:3.6.1")
androidTestImplementation("androidx.compose.ui:ui-test-junit4")
```

3.3 — AndroidManifest.xml

```
xml
```



```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.enterprise.netconfigdiff">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
        android:maxSdkVersion="32" />
    <uses-permission android:name="android.permission.READ_MEDIA_DOCUMENTS" />
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
        android:maxSdkVersion="28" />
    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission android:name="android.permission.VIBRATE" />

    <application
        android:name=".NetConfigApp"
        android:allowBackup="false"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.NetConfigDiff"
        android:networkSecurityConfig="@xml/network_security_config"
        android:usesCleartextTraffic="false">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:windowSoftInputMode="adjustResize"
            android:screenOrientation="portrait">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <provider
            android:name="androidx.core.content.FileProvider"
            android:authorities="${applicationId}.fileprovider"
            android:exported="false"
            android:grantUriPermissions="true">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_paths" />
        </provider>

    </application>
</manifest>

```

Also generate: `res/xml/network_security_config.xml` (allows cleartext traffic only for `10.0.2.2` emulatorlocalhost and `192.168.x.x` LAN,enforcing HTTPS for all other hosts), `res/xml/file_paths.xml` forFileProvider,and `res/values/strings.xml` with all string resources.

3.4 — Theme (Material 3)

Generate a full Material 3 dark/light theme in presentation/theme/ with:

- Primary:  #1565C0 (Enterprise Deep Blue)
- Secondary:  #00897B (Teal — success/safe indicators)
- Error:  #C62828 (Critical risk)
- Warning:  #F57F17 (High risk) — use tertiary slot
- Background:  #0D1117 (dark mode),  #F8FAFC (light mode)
- Surface:  #161B22 (dark),  #FFFFFF (light)
- Custom Typography using FontFamily with Roboto weights 300/400/500/700

3.5 — Screen Specifications

SCREEN 1: SplashScreen.kt

- Animated logo (Lottie or custom Canvas animation) centered on screen
- Auto-checks DataStore for valid JWT token
- If token valid → navigate to Dashboard; else → navigate to Login
- Duration: 2000ms minimum

SCREEN 2: LoginScreen.kt + LoginViewModel.kt

UI Elements:

- App logo + "AI Network Config Diff Reviewer" title
- Email TextField (outlined, keyboard = Email, imeAction = Next)
- Password TextField (outlined, visualTransformation = PasswordVisual, trailing eye icon toggle)
- "Remember Me" checkbox
- "Sign In" FilledButton (full width) — calls POST /api/v1/auth/login
- Loading CircularProgressIndicator overlay during API call
- Error Snackbar for invalid credentials

ViewModel State (LoginUiState sealed class):

- Idle | Loading | Success(token: String, user: UserDto) | Error(message: String)

On Success:

- Store JWT token + user info in EncryptedSharedPreferences via DataStore
- Navigate to Dashboard replacing backstack

Validation:

- Email must match RFC 5322 regex
- Password minimum 8 chars
- Show inline error text under field on validation failure

SCREEN 3: DashboardScreen.kt + DashboardViewModel.kt

Layout: Scaffold with TopAppBar + BottomNavigationBar (5 tabs)

TopAppBar:

- Title: "Dashboard"
- Actions: NotificationsIconButton (badge count), UserAvatarButton → profile sheet

BottomNavigation Tabs:

1. Dashboard (home icon)
2. Upload Config (upload icon)
3. Reviews (list icon)
4. Reports (description icon)
5. Audit Logs (history icon)

Dashboard Body (LazyColumn):

- Summary Cards Row (horizontal scrollable):
 - Total Reviews (count, blue card)
 - Open Reviews (count, amber card)
 - High-Risk Findings (count, red card)
 - Compliance Violations (count, orange card)
 - Pending Approvals (count, purple card)
- "Recent Reviews" section header
- ReviewListItem composables (last 10 reviews, each showing: config type, risk badge, status chip, timestamp, reviewer name)
- FloatingActionButton: "New Review" → navigates to UploadConfig

Data:

- GET /api/v1/dashboard → DashboardStatsDto
- GET /api/v1/reviews?limit=10&sort=created_at:desc → ReviewListDto
- Auto-refresh every 30 seconds using LaunchedEffect + delay loop

SCREEN 4: UploadConfigScreen.kt + UploadConfigViewModel.kt

Layout: Scaffold + Step Indicator (3 steps: Upload → Configure → Submit)

Step 1 — Upload Files:

- "Old Config File" card with drag-drop visual
 - "Pick File" button → `ActivityResultContracts.GetContent` (`mimeType = "*"/*"`)
 - Displays selected filename + file size after selection
 - Supported format badge chips: TF | JSON | YAML | AWS | GCP | AZURE
- "New Config File" card (identical)
- "Config Type" DropdownMenu:
 - Values: AWS_SECURITY_GROUP | AWS_NACL | AWS_ROUTE_TABLE | AWS_VPN | GCP_FIREWALL | GCP_ROUTES | AZURE_NSG | TERRAFORM_IAC | GENERIC_JSON | GENERIC_YAML
- "Cloud Provider" DropdownMenu: AWS | GCP | AZURE | MULTI_CLOUD
- Next Button → validates both files selected → advances to Step 2

Step 2 — Configure Analysis:

- CheckboxGroup "Compliance Frameworks": CIS | NIST | PCI-DSS | Custom Policy
- TextField "Review Title" (required)
- TextField "Change Ticket / JIRA ID" (optional)
- TextField "Description" (multiline, optional)
- SwitchRow "Auto-Approve if Low Risk" (default: off)
- SwitchRow "Notify Manager on Completion" (default: on)
- Back + Submit buttons

Step 3 — Submitting (progress):

- AnimatedLinear ProgressBar
- Step status list (Uploading files → Parsing config → Running diff → AI Analysis → Compliance Check → Generating Report)

- Each step shows: pending / in-progress spinner / completed checkmark / failed

X icon

- On completion: navigate to `DiffViewerScreen` with returned `reviewId`

Backend Call on Submit:

- POST `/api/v1/upload` (multipart: `old_file`, `new_file`, `config_type`, `cloud_provider`)
 - returns `uploadId`
- POST `/api/v1/diff` { `uploadId`, `complianceFrameworks`, `title`, `ticketId`, `description` }
 - returns `reviewId` (async job starts)
- Poll GET `/api/v1/reviews/{reviewId}/status` every 2 seconds until status = COMPLETED | FAILED

SCREEN 5: `DiffViewerScreen.kt` + `DiffViewerViewModel.kt`

Layout: Scaffold with tabs: [ADDED | REMOVED | MODIFIED | SIDE-BY-SIDE]

ADDED tab:

- LazyColumn of DiffLineItem composables
- Each item: green left border, "+" prefix, field name, new value, risk badge

chip

REMOVED tab:

- LazyColumn of DiffLineItem composables
- Each item: red left border, "-" prefix, field name, old value, risk badge chip

MODIFIED tab:

- LazyColumn of DiffLineItem composables
- Each item: amber left border, "~" prefix, field name, old value → new value,

risk badge, expandable AI explanation

SIDE-BY-SIDE tab:

- HorizontalPager with two columns (OLD | NEW)
- Synchronized scrolling
- Color-coded line highlights matching the diff type

RiskBadge composable:

- LOW: green filled chip
- MEDIUM: amber filled chip
- HIGH: orange filled chip
- CRITICAL: red filled chip with pulse animation

FloatingActionButton: "View AI Analysis" → navigates to AIAnalysisScreen

Data: GET /api/v1/diff/{reviewId}/changes → DiffResultDto

SCREEN 6: AIAnalysisScreen.kt + AIAnalysisViewModel.kt

SCREEN 7: ComplianceReportScreen.kt + ComplianceReportViewModel.kt

SCREEN 8: ApprovalWorkflowScreen.kt + ApprovalWorkflowViewModel.kt

Layout: Scaffold with workflow timeline

Timeline Component:

- Vertical step timeline showing: Draft → Pending Review → [Approved / Rejected]
- Closed
- Each step shows: status icon, actor name, timestamp, comment (if any)
- Current step highlighted with primary color

Current Status Banner:

- Large status card: status name + color + description

Reviewer Actions Section (visible if current user has REVIEWER or APPROVER role):

TextField "Add Comment" (multiline, max 500 chars, char counter)

Row of action buttons:

- "Approve" (green, FilledButton) → PATCH /api/v1/reviews/{id}/approve
- "Reject" (red, OutlinedButton) → PATCH /api/v1/reviews/{id}/reject
- "Escalate" (amber, OutlinedButton) → PATCH /api/v1/reviews/{id}/escalate
- "Request Changes" (outlined) → PATCH /api/v1/reviews/{id}/request-changes

Review Summary Card:

- Risk level, compliance score, total findings, AI recommendation

Comments Thread:

- LazyColumn of CommentItem composables (author avatar, name, role, timestamp, comment text)

Data:

- GET /api/v1/reviews/{reviewId}/workflow → WorkflowStatusDto
- PATCH /api/v1/reviews/{id}/approve|reject|escalate { comment }

SCREEN 9: ReportsScreen.kt + ReportsViewModel.kt

Layout: Scaffold + LazyColumn

Report Generation Cards (3 cards):

1. PDF Report Card:

- "Executive Summary" description
- "Generate PDF" FilledButton
- On click: POST /api/v1/report/{reviewId}?format=pdf → triggers background

download

- DownloadManager integration to save to Downloads folder
- "Open" button appears after download; launches PDF viewer intent

2. Markdown Report Card:

- "Developer-Friendly" description
- "Generate Markdown" OutlinedButton
- On click: POST /api/v1/report/{reviewId}?format=markdown → downloads .md

file

- Share intent available

3. JSON Report Card:

- "Machine-Readable / CI-CD Integration" description
- "Export JSON" OutlinedButton
- On click: POST /api/v1/report/{reviewId}?format=json → copies JSON to

clipboard + offers share

Previous Reports Section:

- LazyColumn of past reports for this review
- Each item: format icon, generated timestamp, file size, Download/Share/Delete

actions

WorkManager Integration:

- Large file downloads run as DownloadWorker (WorkManager one-time work request)
- Progress notification in notification drawer with % complete
- Notification on completion with "Open" action

SCREEN 10: AuditLogsScreen.kt + AuditLogsViewModel.kt

Layout: Scaffold with search/filter bar + LazyColumn (Paging 3)

Search/Filter Bar:

- SearchTextField (debounced 300ms)
- FilterChipRow: All | Authentication | Configuration | Analysis | Approval |

Report

- Date range picker (start date / end date)
- "Export Logs" button → downloads CSV

AuditLogItem composable:

- Left colored vertical bar (color by event type)
- Event icon (lock/edit/shield/check/x/file)
- Event title (bold)
- User name + role
- IP address
- Timestamp (relative + absolute)
- Expandable "Details" section (full event payload JSON)

Pagination:

- Paging 3 with RemoteMediator (caches in Room) - Loading footer with CircularProgressIndicator - Pull-to-refresh

Data:

- GET /api/v1/audit?page={page}&size=20&filter={filter}&search={q}&from={date}&to={date}
- PagingSource backed by Room AuditLogEntity + API RemoteMediator

SECTION 4 — ANDROID DATA LAYER: FULL IMPLEMENTATION

4.1 — NetConfigApiService.kt (Retrofit)

Generate ALL endpoints with exact parameter types:

```
kotlin
```

```
interface NetConfigApiService {
```

```
    // Auth
```

```
    @POST("api/v1/auth/login")
```

```
    suspend fun login(@Body request: LoginRequestDto): Response<AuthResponseDto>
```

```
    @POST("api/v1/auth/refresh")
```

```
    suspend fun refreshToken(@Body request: RefreshTokenDto): Response<AuthResponseD
```

```
    @POST("api/v1/auth/logout")
```

```
    suspend fun logout(): Response<Unit>
```

```
    // Dashboard
```

```
    @GET("api/v1/dashboard")
```

```
    suspend fun getDashboardStats(): Response<DashboardStatsDto>
```

```
    // Upload
```

```
    @Multipart
```

```
    @POST("api/v1/upload")
```

```
    suspend fun uploadConfigs(
```

```
        @Part oldFile: MultipartBody.Part,
```

```
        @Part newFile: MultipartBody.Part, @Part("config_type")
```

```
        configType: RequestBody, @Part("cloud_provider") cloudProvider:
```

```
        RequestBody
```

```
    ): Response<UploadResponseDto>
```

```
    // Diff
```

```
    @POST("api/v1/diff")
```

```
    suspend fun createDiff(@Body request: CreateDiffRequestDto): Response<DiffJobDto
```

```
    @GET("api/v1/diff/{reviewId}/changes")
```

```
    suspend fun getDiffChanges(@Path("reviewId") reviewId: String): Response<DiffRes
```

```
    @GET("api/v1/reviews/{reviewId}/status")
```

```
    suspend fun getReviewStatus(@Path("reviewId") reviewId: String): Response<Review
```

```
    // Analysis
```

```
    @POST("api/v1/analyze/{reviewId}")
```

```
    suspend fun triggerAnalysis(@Path("reviewId") reviewId: String): Response<Unit>
```

```
    @GET("api/v1/analyze/{reviewId}")
```

```
    suspend fun getAnalysis(@Path("reviewId") reviewId: String): Response<AIAnalysis
```

```
    // Compliance
```

```
    @POST("api/v1/compliance/{reviewId}")
```

```
    suspend fun runCompliance(@Path("reviewId") reviewId: String): Response<Unit>
```

```
    @GET("api/v1/compliance/{reviewId}")
```

```
    suspend fun getComplianceReport(@Path("reviewId") reviewId: String): Response<Co
```

```
    // Reports
```

```
    @POST("api/v1/report/{reviewId}")
```

```
    suspend fun generateReport(
```

```
        @Path("reviewId") reviewId: String,
```

```
        @Query("format") format: String
```

```
    // pdf | markdown | json
```

```

): Response<ResponseBody>

// Reviews
@GET("api/v1/reviews")
suspend fun getReviews(
    @Query("page") page: Int,
    @Query("size") size: Int,
    @Query("sort") sort: String = "created_at:desc"
): Response<ReviewListDto>

@GET("api/v1/reviews/{reviewId}")
suspend fun getReview(@Path("reviewId") reviewId: String): Response<ReviewDetail>

@GET("api/v1/reviews/{reviewId}/workflow")
suspend fun getWorkflowStatus(@Path("reviewId") reviewId: String): Response<Work

@PATCH("api/v1/reviews/{reviewId}/approve")
suspend fun approveReview(
    @Path("reviewId") reviewId: String,
    @Body request: ApprovalActionDto
): Response<WorkflowStatusDto>

@PATCH("api/v1/reviews/{reviewId}/reject")
suspend fun rejectReview(
    @Path("reviewId") reviewId: String,
    @Body request: ApprovalActionDto
): Response<WorkflowStatusDto>

@PATCH("api/v1/reviews/{reviewId}/escalate")
suspend fun escalateReview(
    @Path("reviewId") reviewId: String,
    @Body request: ApprovalActionDto
): Response<WorkflowStatusDto>

// Audit Logs
@GET("api/v1/audit")
suspend fun getAuditLogs(
    @Query("page") page: Int, @Query("size")
    size: Int, @Query("filter") filter: String?
    @Query("search") search: String? = null,
    @Query("from") fromDate: String? = null,
    @Query("to") toDate: String? = null
): Response<AuditLogListDto>
}

```

4.2 — Room Database

kotlin

```

@Database(
    entities = [
        ReviewEntity::class,
        DiffChangeEntity::class,
        ComplianceFindingEntity::class,
        AuditLogEntity::class,
        UserEntity::class,
        RemoteKeyEntity::class
    ],
    version = 1,
    exportSchema = true
)

@TypeConverters(Converters::class)
abstract class AppDatabase : RoomDatabase() {
    abstract fun reviewDao(): ReviewDao
    abstract fun diffChangeDao(): DiffChangeDao
    abstract fun complianceFindingDao(): ComplianceFindingDao
    abstract fun auditLogDao(): AuditLogDao
    abstract fun userDao(): UserDao
    abstract fun remoteKeyDao(): RemoteKeyDao
}

```

Generate all Entity classes with proper `@Entity`, `@PrimaryKey`, `@ColumnInfo` annotations.
 Generate all DAO interfaces with `@Query`, `@Insert`, `@Update`, `@Delete`, `@Upsert` annotations. All
 DAOs must use `suspend` functions and return `Flow<List<T>>` for observable queries.

4.3 — Domain Models & Use Cases

Generate these Use Cases (each in its own file, injected via constructor, single `invoke()` operator function):

```

LoginUseCase(authRepository: AuthRepository)
GetDashboardStatsUseCase(reviewRepository: ReviewRepository)
UploadConfigFilesUseCase(uploadRepository: UploadRepository)
CreateDiffReviewUseCase(reviewRepository: ReviewRepository)
PollReviewStatusUseCase(reviewRepository: ReviewRepository)
GetDiffChangesUseCase(reviewRepository: ReviewRepository) ← uses Flow + polling
GetAIAnalysisUseCase(analysisRepository: AnalysisRepository)
GetComplianceReportUseCase(complianceRepository: ComplianceRepository)
ApproveReviewUseCase(reviewRepository: ReviewRepository)
RejectReviewUseCase(reviewRepository: ReviewRepository)
EscalateReviewUseCase(reviewRepository: ReviewRepository)
GenerateReportUseCase(reportRepository: ReportRepository)
GetAuditLogsUseCase(auditRepository: AuditRepository)
PagingData<AuditLog> ← returns

```

SECTION 5 — BACKEND: FASTAPI FULL IMPLEMENTATION

5.1 — requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.30.6 sqlalchemy[asyncio]==2.0.35
asynccpg==0.29.0
alembic==1.13.2
pydantic==2.9.1
pydantic-settings==2.5.2
python-multipart==0.0.12
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
deepdiff==8.0.1
pyyaml==6.0.2
python-hcl2==4.3.4
boto3==1.35.13
google-cloud-compute==1.19.2
reportlab==4.2.2
markdown2==2.4.13
celery==5.4.0
redis==5.0.8
httpx==0.27.2
prometheus-fastapi-instrumentator==7.0.0
opentelemetry-sdk==1.27.0
opentelemetry-instrumentation-fastapi==0.48b0
loguru==0.7.2
tenacity==9.0.0
python-dateutil==2.9.0
pytest==8.3.3
pytest-asyncio==0.24.0
pytest-cov==5.0.0
```

5.2 — main.py

```
python
```

```

from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware
from prometheus_fastapi_instrumentator import Instrumentator
from app.api.v1.router import api_router
from app.database import init_db
from app.config import settings
from app.utils.logger import logger

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("Starting AI Network Config Diff Reviewer API...")
    await init_db()
    yield
    logger.info("Shutting down...")

app = FastAPI(
    title="AI Network Config Diff Reviewer",
    description="Enterprise-grade network configuration change review platform",
    version="1.0.0",
    docs_url="/api/docs" if settings.ENVIRONMENT != "production" else None,
    redoc_url="/api/redoc" if settings.ENVIRONMENT != "production" else None,
    lifespan=lifespan
)

# Security Middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
    allow_headers=["Authorization", "Content-Type", "X-Request-ID"],
)

app.add_middleware(TrustedHostMiddleware, allowed_hosts=settings.ALLOWED_HOSTS)

# Prometheus metrics
Instrumentator().instrument(app).expose(app, endpoint="/metrics")

# Router
app.include_router(api_router, prefix="/api/v1")

```

5.3 — config.py (Pydantic Settings)

```
python
```

```

from pydantic_settings import BaseSettings
from typing import List

class Settings(BaseSettings):
    # Application
    APP_NAME: str = "AI Network Config Diff Reviewer"
    ENVIRONMENT: str = "development"
    SECRET_KEY: str
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 30
    REFRESH_TOKEN_EXPIRE_DAYS: int = 7

    # Database
    DATABASE_URL: str = "postgresql+asyncpg://user:pass@host:port/db"
    DATABASE_POOL_SIZE: int = 10
    DATABASE_MAX_OVERFLOW: int = 20

    # Redis
    REDIS_URL: str = "redis://localhost:6379/0"

    # Ollama
    OLLAMA_BASE_URL: str = "http://ollama:11434"
    OLLAMA_MODEL: str = "llama3"
    OLLAMA_TIMEOUT: int = 120
    OLLAMA_MAX_RETRIES: int = 3

    # CORS
    ALLOWED_ORIGINS: List[str] = ["http://localhost:3000"]
    ALLOWED_HOSTS: List[str] = ["*"]

    # File Upload
    MAX_UPLOAD_SIZE_MB: int = 50
    UPLOAD_DIR: str = "/tmp/uploads"

    # Report Output
    REPORT_OUTPUT_DIR: str = "/tmp/reports"

    class Config:
        env_file = ".env"
        case_sensitive = True

settings = Settings()

```

5.4 — database.py

```
python
```

```

from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionm
from sqlalchemy.orm import DeclarativeBase
from app.config import settings

engine = create_async_engine(
    settings.DATABASE_URL,
    pool_size=settings.DATABASE_POOL_SIZE,
    max_overflow=settings.DATABASE_MAX_OVERFLOW,
    pool_pre_ping=True,
    pool_recycle=3600,
    echo=settings.ENVIRONMENT == "development"
)

AsyncSessionLocal = async_sessionmaker(
    engine, class_=AsyncSession, expire_on_commit=False
)

class Base(DeclarativeBase):
    pass

async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

async def get_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
        finally:
            await session.close()

```

5.5 — PostgreSQL Database Schema (SQLAlchemy Models)

Generate ALL models with full column definitions, indexes, and relationships:

```
python
```



```
# models/review.py
```

```
class Review(Base):
    __tablename__ = "reviews"
    id: uuid (PK, default=uuid4)
    title: str (not null)
    ticket_id: str (nullable)
    description: text (nullable)
    config_type: str (enum: AWS_SECURITY_GROUP | AWS_NACL | etc.)
    cloud_provider: str (enum: AWS | GCP | AZURE | MULTI_CLOUD)
    status: over all (enum: DRAFT | PENDING_REVIEW | APPROVED | REJECTED | ESCALATED | CL
overall_risk_score: str (enum: LOW | MEDIUM | HIGH | CRITICAL | UNKNOWN)
float (nullable, 0.0-100.0)
compliance_score: float (nullable, 0.0-100.0)
ai_summary: text (nullable)
ai_recommendation: str (enum: APPROVE | REJECT | ESCALATE | nullable)
auto_approve_if_low_risk: bool (default=False)
notify_manager: bool (default=True) (not
old_config_path: str null) (not null)
new_config_path: str
compliance_frameworks: ARRAY[str] (default=[])
created_by: uuid (FK → users.id)
assigned_reviewer: uuid (FK → users.id, nullable)
created_at: datetime (server_default=now(), indexed)
updated_at: datetime (onupdate=now())
completed_at: datetime (nullable)
```

```
# models/diff_change.py
```

```
class DiffChange(Base):
    __tablename__ = "diff_changes"
    id: uuid (PK)
    review_id: uuid (FK → reviews.id, CASCADE DELETE, indexed)
    change_type: str (enum: ADDED | REMOVED | MODIFIED)
    field_path: str (not null) # e.g. "rules[0].source_ranges"
    field_name: str (not null) # human-readable field name
    old_value: text (nullable)
    new_value: text (nullable)
    risk_level: str (enum: LOW | MEDIUM | HIGH | CRITICAL)
    risk_score: float
    ai_explanation: text (nullable)
    affected_resource: str (nullable)
    cis_control_ref: str (nullable)
    nist_control_ref: str (nullable)
    order_index: int
```

```
# models/compliance_finding.py
```

```
class ComplianceFinding(Base):
    __tablename__ = "compliance_findings"
    id: uuid (PK)
    review_id: uuid (FK → reviews.id, CASCADE DELETE, indexed)
    framework: str (enum: CIS | NIST | PCI_DSS | CUSTOM)
    control_id: str # e.g. "CIS-4.1", "NIST-AC-17"
    control_name: str
    status: str (enum: PASS | FAIL | WARNING | NOT_APPLICABLE)
    finding_description: text
    remediation_guidance: text (nullable)
```

evidence: text (nullable)
severity: **str** (enum: LOW | MEDIUM | HIGH | CRITICAL)

models/user.py

```
class User(Base):
    __tablename__ = "users"
    id: uuid (PK)
    email: str (unique, not null, indexed)
    hashed_password: str (not null)
    full_name: str (not null)
    role: str (enum: NETWORK_ENGINEER | REVIEWER | APPROVER | ADMIN)
    is_active: bool (default=True)
    is_verified: bool (default=False)
    created_at: datetime
    last_login: datetime (nullable)
```

models/audit.py

```
class AuditLog(Base):
    __tablename__ = "audit_logs"
    id: uuid (PK)
    event_type: str (enum: AUTH_LOGIN | AUTH_LOGOUT | CONFIG_UPLOAD | DIFF_CREATED | ANALYSIS_COMPLETED | COMPLIANCE_CHECKED | REVIEW_APPROVED | REVIEW_REJECTED | REVIEW_ESCALATED | REPORT_GENERATED | US
    event_description: str
    user_id: uuid (FK → users.id, nullable)
    user_email: str (nullable)
    user_role: str (nullable)
    review_id: uuid (FK → reviews.id, nullable)
    ip_address: str (nullable)
    user_agent: str (nullable)
    request_id: str (nullable)
    payload: JSON (nullable)
    created_at: datetime (server_default=now(), indexed)
```

models/workflow_step.py

```
class WorkflowStep(Base):
    __tablename__ = "workflow_steps"
    id: uuid (PK)
    review_id: uuid (FK → reviews.id, CASCADE DELETE, indexed)
    step_number: int
    status: str (enum: DRAFT | PENDING_REVIEW | APPROVED | REJECTED | ESCALATED | CL
    actor_id: uuid (FK → users.id, nullable)
    actor_name: str (nullable)
    actor_role: str (nullable)
    comment: text (nullable)
    created_at: datetime
```

5.6 — diff_engine.py (DeepDiff Integration)

python

```
"""
```

Full implementation of the configuration diff engine.

Supports: AWS Security Groups, GCP Firewall Rules, Azure NSGs, Terraform HCL, JSON, """

```
import json
```

```
import yaml
```

```
import hcl2
```

```
from deepdiff import DeepDiff
```

```
from typing import Any, Dict, List, Tuple
```

```
from app.domain.models import DiffChange, ChangeType, RiskLevel
```

```
class ConfigDiffEngine:
```

```
    SENSITIVE_PORTS = {22, 3389, 3306, 5432, 6379, 9200, 27017, 5984, 6443, 8080, 84
```

```
    CRITICAL_CIDRS = {"0.0.0.0/0", "::/0"}
```

```
    INTERNAL_CIDR_PREFIXES = ("10.", "172.16.", "172.17.", "172.18.", "172.19.",  
                              "172.20.", "172.21.", "172.22.", "172.23.", "172.24.",  
                              "172.25.", "172.26.", "172.27.", "172.28.", "172.29.",  
                              "172.30.", "172.31.", "192.168.")
```

```
    def parse_config(self, file_content: str, config_type: str) -> Dict:
```

```
        """Parse configuration file based on detected type."""
```

```
        config_type = config_type.upper()
```

```
        if config_type in ("GENERIC_JSON", "AWS_SECURITY_GROUP", "AWS_NACL",  
                           "AWS_ROUTE_TABLE", "AWS_VPN", "GCP_FIREWALL",  
                           "GCP_ROUTES", "AZURE_NSG"):
```

```
            return json.loads(file_content)
```

```
        elif config_type in ("GENERIC_YAML", "AWS_TRANSIT_GATEWAY"):
```

```
            return yaml.safe_load(file_content)
```

```
        elif config_type == "TERRAFORM_IAC":
```

```
            return hcl2.loads(file_content)
```

```
        else:
```

```
            # Try JSON first, then YAML, then HCL2
```

```
            for parser in [json.loads, yaml.safe_load]:
```

```
                try:
```

```
                    return parser(file_content)
```

```
                except Exception:
```

```
                    continue
```

```
            return hcl2.loads(file_content)
```

```
    def compute_diff(self, old_config: Dict, new_config: Dict) -> List[DiffChange]:
```

```
        """
```

Compute structural diff between two config dictionaries.

Returns list of DiffChange objects with risk scoring.

```
        """
```

```
        changes: List[DiffChange] = []
```

```
        diff = DeepDiff(old_config, new_config, ignore_order=True, verbose_level=2)
```

```
        #Process added items
```

```
        for path, value in diff.get("dictionary_item_added", {}).items():  
            change = self._process_change(path, None, value, ChangeType.ADDED)  
            changes.append(change)
```

```
        # Process removed items
```

```
        for path, value in diff.get("dictionary_item_removed", {}).items():
```

```

        change = self._process_change(path, value, None, ChangeType.REMOVED)
        changes.append(change)

#Processmodified values
for path, change_info in diff.get("values_changed", {}).items():
    change = self._process_change(
        path, change_info.get("old_value"),
        change_info.get("new_value"),
        ChangeType.MODIFIED

    )
    changes.append(change)

#Processiterable additions/removals
for path, items in diff.get("iterable_item_added", {}).items():
    for item in (items if isinstance(items, list) else [items]):
        change = self._process_change(path, None, str(item), ChangeType.ADDE
        changes.append(change)

for path, items in diff.get("iterable_item_removed", {}).items():
    for item in (items if isinstance(items, list) else [items]):
        change = self._process_change(path, str(item), None, ChangeType.REMO
        changes.append(change)

return sorted(changes, key=lambda c: c.risk_score, reverse=True)

def _process_change(self, path: str, old_value: Any, new_value: Any,
                    change_type: ChangeType) -> DiffChange:
    """Classify achange and assign initial rule-based risk score."""
    field_name = self._extract_field_name(path)
    risk_level, risk_score = self._compute_risk(field_name, old_value, new_value)
    return DiffChange(
        field_path=str(path),
        field_name=field_name,
        old_value=str(old_value) if old_value is not None else None,
        new_value=str(new_value) if new_value is not None else None,
        change_type=change_type,
        risk_level=risk_level,
        risk_score=risk_score,
        affected_resource=self._extract_resource_name(path)
    )

def _compute_risk(self, field_name: str, old_value: Any, new_value: Any,
                  change_type: ChangeType) -> Tuple[RiskLevel, float]:
    """Rule-basedrisk scoring engine."""
    field_lower = field_name.lower()
    new_val_str = str(new_value).lower() if new_value else ""
    old_val_str = str(old_value).lower() if old_value else ""

    # CRITICAL: Exposure to public internet
    if new_value and any(cidr in new_val_str for cidr in ["0.0.0.0/0", "::/0"]):
        if old_value and not any(cidr in old_val_str for cidr in ["0.0.0.0/0",
        return RiskLevel.CRITICAL, 95.0
    elif change_type == ChangeType.ADDED:
        return RiskLevel.CRITICAL, 90.0

```

```

# CRITICAL: Sensitive port exposure
if any(k in field_lower for k in ["port", "from_port", "to_port", "destination"])
    try:
        port = int(str(new_value).split("-")[0].strip())
        if port in self.SENSITIVE_PORTS and new_val_str in ["0.0.0.0/0", ":::"]
            return RiskLevel.CRITICAL, 98.0
        elif port in self.SENSITIVE_PORTS:
            return RiskLevel.HIGH, 75.0
    except ValueError:
        pass

# HIGH: Any CIDR expansion
if any(k in field_lower for k in ["cidr", "source_range", "ip_range", "prefix"])
    if old_value and new_value:
        if self._is_cidr_expansion(old_val_str, new_val_str):
            return RiskLevel.HIGH, 70.0

# HIGH: Protocol changes (TCP → ALL)
if "protocol" in field_lower:
    if new_val_str in ["-1", "all", "*", "any"] and old_val_str not in ["-1"]
        return RiskLevel.HIGH, 72.0

# HIGH: VPN/encryption changes
if any(k in field_lower for k in ["vpn", "encryption", "psk", "ike", "tunnel"])
    return RiskLevel.HIGH, 68.0

# MEDIUM: Route changes
if any(k in field_lower for k in ["route", "next_hop", "gateway"]):
    return RiskLevel.MEDIUM, 45.0

# MEDIUM: Security group/firewall rule changes
if any(k in field_lower for k in ["rule", "policy", "acl", "firewall"]):
    return RiskLevel.MEDIUM, 40.0

# LOW: Tag, description, name changes
if any(k in field_lower for k in ["tag", "label", "name", "description", "comment"])
    return RiskLevel.LOW, 10.0

# DEFAULT: LOW
return RiskLevel.LOW, 20.0

def _is_new_cidr_broader(self, old_cidr, new_cidr) -> bool:
    """Check if new CIDR is broader than old CIDR."""
    try:
        import ipaddress
        old_net = ipaddress.ip_network(old_cidr, strict=False)
        new_net = ipaddress.ip_network(new_cidr, strict=False)
        return new_net.prefixlen < old_net.prefixlen
    except ValueError:
        return False

def _extract_field_name(self, path: str) -> str:
    """Convert DeepDiff path to human-readable field name."""
    import re

```

```
cleaned = re.sub(r"root\[", "", str(path))
cleaned = re.sub(r"\\", ".", cleaned)
cleaned = re.sub(r"[\\]", "", cleaned)
return cleaned
```

```
def _extract_resource_name(self, path: str) -> str:
    """Extract the resource name from path."""
    parts = str(path).split("/")
    if len(parts) > 1:
        return parts[1].strip("/")
    return "Unknown Resource"
```

5.7 — ollama_service.py (Llama 3 via Ollama)

python

.....

Production Ollama + Llama 3 integration for network config change analysis.
Implements retry logic, prompt templating, and structured JSON response parsing.

.....

```
import httpx
import json
from tenacity import retry, stop_after_attempt, wait_exponential, retry_if_exception
from app.config import settings
from app.utils.logger import logger
from typing import List
from app.domain.models import DiffChange, AIAnalysisResult
```

SYSTEM_PROMPT = """You are a senior network security engineer and compliance expert 15 years of experience reviewing network configuration changes across AWS, GCP, Azur and Terraform IaC environments. You provide precise, actionable security analysis.

You MUST respond ONLY with a valid JSON object matching this exact schema:

```
{
  "overall_risk_level": "LOW|MEDIUM|HIGH|CRITICAL",
  "overall_risk_score": <float 0.0-100.0>,
  "ai_recommendation": "APPROVE|REJECT|ESCALATE",
  "executive_summary": "<2-3 sentence plain English summary>",
  "security_impact": "<detailed security impact analysis>",
  "findings": [
    {
      "title": "<finding title>",
      "description": "<plain English explanation>",
      "affected_resource": "<resource name>",
      "risk_level": "LOW|MEDIUM|HIGH|CRITICAL",
      "cis_control_ref": "<e.g. CIS-4.1 or null>",
      "nist_control_ref": "<e.g. NIST-AC-17 or null>",
      "recommendation": "<specific actionable recommendation>"
    }
  ],
  "recommendations": [
    {
      "priority": 1,
      "recommendation": "<actionable recommendation>",
      "implementation_guidance": "<how to implement>"
    }
  ],
  "change_explanations": {
    "field_path": "<plain English explanation of this specific change>"
  }
}
```

```
class OllamaService:

    def __init__(self):
        self.client = httpx.AsyncClient(
            base_url=settings.OLLAMA_BASE_URL,
            timeout=httpx.Timeout(settings.OLLAMA_TIMEOUT)
        )

    @retry(
```

```

stop=stop_after_attempt(settings.OLLAMA_MAX_RETRIES),
wait=wait_exponential(multiplier=1, min=2, max=30),
retry=retry_if_exception_type((httpx.ConnectError, httpx.TimeoutException))
)
async def analyze_changes(self, changes: List[DiffChange],
                           config_type: str) -> AIAnalysisResult:
    """Send diffchanges to Llama 3 for intelligent security analysis."""

    changes_payload = [
        {
            "field_path": c.field_path,
            "field_name": c.field_name, "change_type":
            c.change_type.value, "old_value": c.old_value,
            "new_value": c.new_value,
            "initial_risk_level": c.risk_level.value,
            "initial_risk_score": c.risk_score,

        }
        for c in changes
    ]

```

user_prompt = f"""Analyze the following network configuration changes for
 a {config_type} configuration and provide a comprehensive security review:

CONFIGURATION CHANGES:

```
{json.dumps(changes_payload, indent=2)}
```

Analyze each change for:

1. Security implications and attack surface expansion
2. Public internet exposure risks (any 0.0.0.0/0 or ::/0 changes are CRITICAL)
3. Sensitive port exposure (SSH:22, RDP:3389, MySQL:3306, PostgreSQL:5432, Redis:637)
4. Compliance violations (CIS AWS Foundations, NIST SP 800-53, PCI-DSS)
5. Traffic routing impact
6. VPN/encryption configuration integrity

Provide plain English explanations that a non-technical manager can understand.

Your recommendation should be APPROVE (safe changes), REJECT (security risks found), or ESCALATE (requires senior security team review).

Respond ONLY with the JSON object as specified. No markdown, no explanations outside

```
logger.info(f"Sending {len(changes)} changes to Ollama Llama3 for analysis")
```

```

response = await self.client.post(
    "/api/generate",
    json={
        "model": settings.OLLAMA_MODEL,
        "system": SYSTEM_PROMPT,
        "prompt": user_prompt,
        "stream": False,
        "format": "json",
        "options": {
            "temperature": 0.1,
            "top_p": 0.9,
            "num_ctx": 8192
        }
    }
)

```



```

        }
    }
)
response.raise_for_status()
result = response.json()
raw_text = result.get("response", "")

# Parse and validate JSON response
try:
    analysis_data = json.loads(raw_text)
    return AIAnalysisResult(**analysis_data)
except (json.JSONDecodeError, ValueError) as e:
    logger.error(f'Ollama response parsing failed: {e}. Raw: {raw_text[:500]}')
    #Return safe fallback analysis
    return self._fallback_analysis(changes)

def _fallback_analysis(self, changes: List[DiffChange]) -> AIAnalysisResult:
    """Rule-based fallback when Ollama is unavailable."""
    critical_changes = [c for c in changes if c.risk_level.value == "CRITICAL"]
    high_changes = [c for c in changes if c.risk_level.value == "HIGH"]
    overall = "CRITICAL" if critical_changes else "HIGH" if high_changes else "M
    recommendation = "REJECT" if critical_changes else "ESCALATE" if high_change
    return AIAnalysisResult(
        overall_risk_level=overall,
        overall_risk_score=max((c.risk_score for c in changes), default=0.0),
        ai_recommendation=recommendation,
        executive_summary="AI analysis unavailable. Rule-based scoring applied.
        security_impact="Unable to perform deep AI analysis. Please review chang
        findings=[],
        recommendations=[{"priority": 1, "recommendation": "Manual review requir
                        "implementation_guidance": "Escalate to senior securit
        change_explanations={}
    )

async def close(self):
    await self.client.aclose()

```

5.8 — compliance_engine.py

python

.....

Enterprise compliance validation engine.

Validates network config changes against CIS, NIST SP 800-53, PCI-DSS, and custom po

.....

from typing import List, Dict

from app.domain.models import DiffChange, ComplianceFinding, ComplianceFramework, Fi

class ComplianceEngine:

```
CIS_RULES: Dict[str, dict] = {
    "CIS-4.1": {
        "name": "Ensure no security groups allow ingress from 0.0.0.0/0 to port",
        "check": lambda c: not (c.new_value and "0.0.0.0/0" in str(c.new_value)
                                and any(p in str(c.field_name).lower() for p in
                                        "22", "3389", "443", "5424", "5900", "5936", "6443", "8443")),
        "severity": "CRITICAL",
        "remediation": "Remove or restrict SSH access to specific trusted IP ranges.",
    },
    "CIS-4.2": {
        "name": "Ensure no security groups allow ingress from 0.0.0.0/0 to port",
        "check": lambda c: not (c.new_value and "0.0.0.0/0" in str(c.new_value)
                                and any(p in str(c.field_name).lower() for p in
                                        "22", "3389", "443", "5424", "5900", "5936", "6443", "8443")),
        "severity": "CRITICAL",
        "remediation": "Remove or restrict RDP access. Use VPN or bastion host if needed.",
    },
    "CIS-4.3": {
        "name": "Ensure VPC flow logging is enabled",
        "check": lambda c: "flow_log" not in str(c.field_name).lower() or c.new_value != "true",
        "severity": "MEDIUM",
        "remediation": "Enable VPC flow logging for all VPCs.",
    },
    "CIS-5.1": {
        "name": "Ensure routing tables do not have unrestricted routes",
        "check": lambda c: not (c.new_value and "0.0.0.0/0" in str(c.new_value)
                                and "route" in str(c.field_name).lower()),
        "severity": "HIGH",
        "remediation": "Review route table entries and restrict default routes.",
    }
}

NIST_RULES: Dict[str, dict] = {
    "NIST-AC-17": {
        "name": "Remote Access Control",
        "check": lambda c: not any(p in str(c.field_name).lower() for p in ["22", "3389", "443", "5424", "5900", "5936", "6443", "8443"])
                    or "0.0.0.0/0" not in str(c.new_value or ""),
        "severity": "HIGH",
        "remediation": "Implement multi-factor authentication for remote access.",
    },
    "NIST-SC-7": {
        "name": "Boundary Protection",
        "check": lambda c: "0.0.0.0/0" not in str(c.new_value or ""),
        "severity": "CRITICAL",
        "remediation": "Implement boundary protection mechanisms at all network boundaries.",
    },
    "NIST-SI-4": {
        "name": "Information System Monitoring",
    }
```

```

        "check": lambda c: "monitoring" not in str(c.field_name).lower() or c.ne
        "severity": "MEDIUM",
        "remediation": "Ensure monitoring configurations are not disabled."
    }
}

PCI_RULES: Dict[str, dict] = {
    "PCI-1.2.1": {
        "name": "Restrict inbound and outbound traffic to that required",
        "check": lambda c: "0.0.0.0/0" not in str(c.new_value or "")
            or any(p in str(c.field_name).lower() for p in ["3306",
        "severity": "CRITICAL",
        "remediation": "Database ports must not be exposed to 0.0.0.0/0."
    },
    "PCI-1.3.2": {
        "name": "Restrict inbound internet traffic to IP addresses within DMZ",
        "check": lambda c: "::/0" not in str(c.new_value or ""),
        "severity": "HIGH",
        "remediation": "Restrict IPv6 access. Do not allow ::/0 in firewall rule
    }
}

CUSTOM_RULES: List[dict] = [
    {
        "id": "CUSTOM-001",
        "name": "No database port may be exposed to 0.0.0.0/0",
        "check": lambda c: not (
            any(p in str(c.field_name).lower() for p in ["3306", "5432", "27017"
            and "0.0.0.0/0" in str(c.new_value or ""))
        ),
        "severity": "CRITICAL",
        "remediation": "Database access must be restricted to application subnet
    },
    {
        "id": "CUSTOM-002",
        "name": "VPN configurations must use AES-256 encryption",
        "check": lambda c: "vpn" not in str(c.field_name).lower()
            or "aes256" in str(c.new_value or "").lower()
            or "aes-256" in str(c.new_value or "").lower(),
        "severity": "HIGH",
        "remediation": "Update VPN configuration to use AES-256 encryption algo
    }
]

def validate(self, changes: List[DiffChange],
              frameworks: List[str]) -> List[ComplianceFinding]:
    findings: List[ComplianceFinding] = []
    framework_set = {f.upper() for f in frameworks}

    if "CIS" in framework_set:
        findings.extend(self._check_framework(changes, self.CIS_RULES, "CIS"))
    if "NIST" in framework_set:
        findings.extend(self._check_framework(changes, self.NIST_RULES, "NIST"))
    if "PCI_DSS" in framework_set or "PCI-DSS" in framework_set:
        findings.extend(self._check_framework(changes, self.PCI_RULES, "PCI_DSS")

```

```

if "CUSTOM" in framework_set:
    findings.extend(self._check_custom(changes))

return findings

def _check_framework(self, changes: List[DiffChange], rules: Dict,
                    framework: str) -> List[ComplianceFinding]:
    findings = []
    for control_id, rule in rules.items():
        failing_changes = [c for c in changes if not rule["check"](c)]
        if failing_changes:
            findings.append(ComplianceFinding(
                framework=framework,
                control_id=control_id,
                control_name=rule["name"],
                status=FindingStatus.FAIL,
                finding_description=f"Change violates {control_id}: {rule['name']}"
                f"Affected fields: {' '.join(c.field_name for c in failing_changes)}",
                remediation_guidance=rule["remediation"],
                evidence=f"Found {len(failing_changes)} change(s) violating this"
                severity=rule["severity"]
            ))
        else:
            findings.append(ComplianceFinding(
                framework=framework,
                control_id=control_id,
                control_name=rule["name"],
                status=FindingStatus.PASS,
                finding_description=f"All changes comply with {control_id}.",
                remediation_guidance=None,
                evidence=f"Checked {len(changes)} change(s). No violations found"
                severity=rule["severity"]
            ))
    return findings

def _check_custom(self, changes: List[DiffChange]) -> List[ComplianceFinding]:
    findings = []
    for rule in self.CUSTOM_RULES:
        failing = [c for c in changes if not rule["check"](c)]
        status = FindingStatus.FAIL if failing else FindingStatus.PASS
        findings.append(ComplianceFinding(
            framework="CUSTOM",
            control_id=rule["id"],
            control_name=rule["name"],
            status=status,
            finding_description=rule["name"] if not failing else
                f"VIOLATION: {rule['name']}. Affected: {' '.join(c.field_name for c in failing)}",
            remediation_guidance=rule["remediation"],
            evidence=f"{len(failing)} violation(s) found." if failing else "No v
            severity=rule["severity"]
        ))
    return findings

```

```
python
```

.....

Enterprise report generation service.

Produces executive-grade PDF reports using ReportLab,
developer-friendly Markdown, and machine-readable JSON.

.....

```
import json
import os
from datetime import datetime
from typing import Optional
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch, mm
from reportlab.platypus import (SimpleDocTemplate, Paragraph, Spacer, Table,
                                TableStyle, HRFlowable, Image, KeepTogether)
from reportlab.lib.enums import TA_LEFT, TA_CENTER, TA_RIGHT
from app.domain.models import ReviewDetail, RiskLevel
```

```
RISK_COLORS = {
    "LOW": colors.HexColor("#1B8A2C"),
    "MEDIUM": colors.HexColor("#D97706"),
    "HIGH": colors.HexColor("#D95D0A"),
    "CRITICAL": colors.HexColor("#C62828"),
}
```

```
class ReportService:
```

```
    def generate_pdf(self, review: ReviewDetail, output_path: str) -> str:
        """Generate executive-grade PDF report."""
        doc = SimpleDocTemplate(
            output_path,
            pagesize=A4,
            rightMargin=20*mm,
            leftMargin=20*mm,
            topMargin=25*mm,
            bottomMargin=20*mm,
            title=f"Network Config Review: {review.title}",
            author="AI Network Config Diff Reviewer"
        )
        styles = getSampleStyleSheet()
        story = []

        #Cover Section
        story.extend(self._build_cover(review, styles))
        story.append(Spacer(1, 0.3*inch))

        #Executive Summary
        story.extend(self._build_executive_summary(review, styles))
        story.append(Spacer(1, 0.2*inch))

        #Risk Assessment Table
        story.extend(self._build_risk_table(review, styles))
        story.append(Spacer(1, 0.2*inch))

        # Diff Changes Table
```

```

story.extend(self._build_changes_table(review, styles))
story.append(Spacer(1, 0.2*inch))

#Compliance Findings
story.extend(self._build_compliance_section(review, styles))
story.append(Spacer(1, 0.2*inch))

#AI Recommendations
story.extend(self._build_recommendations(review, styles))
story.append(Spacer(1, 0.2*inch))

#Approval Section
story.extend(self._build_approval_section(review, styles))

doc.build(story, onFirstPage=self._header_footer, onLaterPages=self._header_
return output_path

def _build_cover(self, review: ReviewDetail, styles) -> list:
    """Build report header/cover block."""
    risk_color = RISK_COLORS.get(review.overall_risk_level, colors.gray)
    title_style = ParagraphStyle("Title", parent=styles["Title"],
                                fontSize=20, textColor=colors.HexColor("#1565C0"),
                                spaceAfter=6)

    elements = [
        Paragraph("AINetworkConfigDiffReviewer", title_style),
        Paragraph(f"<b>Review:</b> {review.title}", styles["Normal"]),
        Paragraph(f"<b>ConfigType:</b> {review.config_type.replace('_', ' ').title()}"
                  styles["Normal"]),
        Paragraph(f"<b>CloudProvider:</b> {review.cloud_provider}", styles["Normal"]),
        Paragraph(f"<b>ReviewID:</b> {review.id}", styles["Normal"]),
        Paragraph(f"<b>Generated:</b> {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}"
                  styles["Normal"]),
        Spacer(1, 0.1*inch),
        HRule(width="100%", thickness=2, color=risk_color),
        Paragraph(
            f"<fontcolor='{risk_color.hexval()[1:]}'><b>OVERALL RISK: "
            f"{review.overall_risk_level} (Score: {review.overall_risk_score})"
            ParagraphStyle("Risk", parent=styles["Normal"], fontSize=14, align="center",
            ),
        Paragraph(
            f"<b>AI Recommendation: {review.ai_recommendation}</b> | "
            f"<b>Approval Status: {review.status}</b>'",
            ParagraphStyle("Status", parent=styles["Normal"], fontSize=11, align="center",
            ),
        HRule(width="100%", thickness=2, color=risk_color),
    ]
    return elements

def _header_footer(self, canvas, doc):
    """Page header and footer."""
    canvas.saveState()
    canvas.setFillColor(colors.HexColor("#1565C0"))
    canvas.setFont("Helvetica-Bold", 8)
    canvas.drawString(20*mm, 287*mm, "AI Network Config Diff Reviewer — CONFIDENTIAL")
    canvas.drawRightString(190*mm, 287*mm, "Page 1 of 1")
    canvas.restoreState()

```

```

                                datetime.now().strftime("%Y-%m-%d"))
canvas.setFont("Helvetica", 8)
canvas.drawString(20*mm, 10*mm, f"Page {doc.page}")
canvas.drawRightString(190*mm, 10*mm, "Generated by AI Network Config Diff R
canvas.restoreState()

def generate_markdown(self, review: ReviewDetail) -> str:
    """Generate developer-friendly Markdown report."""
    lines = [
        f"# Network Config Review Report",
        f"""
        f"""Review Title:** {review.title} ",
        f"""Review ID:** `{review.id}` ",
        f"""Config Type:** {review.config_type} ",
        f"""Cloud Provider:** {review.cloud_provider} ",
        f"""Generated:** {datetime.now().isoformat()}Z ",
        f"""Ticket ID:** {review.ticket_id or 'N/A'} ",
        f"""
        f"---",
        f"""
        f"""### Risk Assessment",
        f"""
        f"| Attribute | Value |",
        f"|-----|-----|",
        f"| Overall Risk Level | **{review.overall_risk_level}** |",
        f"| Risk Score | {review.overall_risk_score:.1f} / 100 |",
        f"| Compliance Score | {review.compliance_score:.1f}% |",
        f"| AI Recommendation | **{review.ai_recommendation}** |",
        f"| Approval Status | {review.status} |",
        f"""
        f"---",
        f"""
        f"""### Executive Summary",
        f"""
        review.ai_summary or "_AI summary not available._",
        f"""
        f"---",
        f"""
        f"""### Configuration Changes ({len(review.diff_changes)} total)",
        f"""
        f"| # | Field | Change Type | Old Value | New Value | Risk |",
        f"|---|-----|-----|-----|-----|-----|",

    ]

    for i, c in enumerate(review.diff_changes, 1):
        old_v = (c.old_value or "_N/A_")[:60]
        new_v = (c.new_value or "_N/A_")[:60]
        lines.append(
            f"| {i} | {c.field_name[:40]} | {c.change_type} | "
            f"{old_v} | {new_v} | **{c.risk_level}** |"
        )

    lines.extend([
        f"""
        f"---",
        f"""
        f"""### Compliance Findings",

```



```

f"""
    ])
    for framework in ["CIS", "NIST", "PCI_DSS", "CUSTOM"]:
        fw_findings = [f for f in review.compliance_findings if f.framework == f
        if fw_findings:
            lines.append(f'### {framework} Controls')
            lines.append(f'')
            for f in fw_findings:
                status_emoji = "✅" if f.status == "PASS" else "❌" if f.status
                lines.append(f'- {status_emoji} {f.control_id} — {f.control_
                if f.status != "PASS":
                    lines.append(f' - **Finding:** {f.finding_description}')
                    lines.append(f' - **Remediation:** {f.remediation_guidance}
            lines.append(f'')

lines.extend([
    f'---',
    f' ',
    f'## Approval Workflow',
    f' ',
    f'| Step | Status | Actor | Timestamp | Comment |',
    f'|-----|-----|-----|-----|-----|',
])
for step in review.workflow_steps:
    lines.append(
        f'| {step.step_number} | {step.status} | {step.actor_name or
        f'| {step.created_at} | {step.comment or
    )

lines.extend([
    f' ',
    f'---',
    f' ',
    f'_Report generated by AI Network Config Diff Reviewer v1.0 — Powered by

])
return "\n".join(lines)

def generate_json(self, review: ReviewDetail) -> dict:
    """Generate machine-readable JSON export."""
    return {
        "schema_version": "1.0",
        "generated_at": datetime.now().isoformat() + "Z",
        "review": {
            "id": str(review.id),
            "title": review.title,
            "config_type": review.config_type,
            "cloud_provider": review.cloud_provider,
            "ticket_id": review.ticket_id,
            "status": review.status,
            "overall_risk_level": review.overall_risk_level,
            "overall_risk_score": review.overall_risk_score,
            "compliance_score": review.compliance_score,
            "ai_recommendation": review.ai_recommendation,
            "ai_summary": review.ai_summary,
            "compliance_frameworks": review.compliance_frameworks,
            "created_at": review.created_at.isoformat() + "Z",

```

```

        "completed_at": review.completed_at.isoformat() + "Z"
        if review.completed_at else None,
    },
    "diff_changes": [
        {
            "field_path": c.field_path, "field_name": c.field_name,
            "change_type": c.change_type,
            "old_value": c.old_value,
            "new_value": c.new_value,
            "risk_level": c.risk_level,
            "risk_score": c.risk_score,
            "ai_explanation": c.ai_explanation,
            "cis_control_ref": c.cis_control_ref,
            "nist_control_ref": c.nist_control_ref,

        }
        for c in review.diff_changes
    ],
    "compliance_findings": [
        {
            "framework": f.framework,
            "control_id": f.control_id,
            "control_name": f.control_name,
            "status": f.status,
            "severity": f.severity,
            "finding": f.finding_description,
            "remediation": f.remediation_guidance,

        }
        for f in review.compliance_findings
    ],
    "workflow": [
        {
            "step": s.step_number,
            "status": s.status,
            "actor": s.actor_name,
            "role": s.actor_role,
            "comment": s.comment,
            "timestamp": s.created_at.isoformat() + "Z",

        }
        for s in review.workflow_steps
    ]
}

```

SECTION 6 — DOCKER: FULL CONTAINERIZATION

6.1 — backend/Dockerfile

dockerfile

FROM python:3.12-slim

LABEL maintainer="enterprise@netconfigdiff.com"

LABEL description="AI Network Config Diff Reviewer — FastAPI Backend"

Security: run as non-root

RUN groupadd -r appgroup && useradd -r -g appgroup appuser

Install system dependencies

RUN apt-get update && apt-get install -y --no-install-recommends \
 curl \
 libpq-dev \
 gcc \
 && apt-get clean \
 && rm -rf /var/lib/apt/lists/*

WORKDIR /app

Install Python dependencies (cached layer)

COPY requirements.txt .

RUN pip install --no-cache-dir --upgrade pip && \
 pip install --no-cache-dir -r requirements.txt

Copy application source

COPY . .

Create upload and report directories

RUN mkdir -p /tmp/uploads /tmp/reports && \
 chown -R appuser:appgroup /tmp/uploads /tmp/reports /app

USER appuser

EXPOSE 8000

HEALTHCHECK--interval=30s --timeout=10s --start-period=15s --retries=3 \
 CMD curl -f http://localhost:8000/api/v1/health || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", \
 "--workers", "4", "--loop", "uvloop", "--http", "httptools"]

6.2 — docker-compose.yml (COMPLETE)

yaml

version: "3.9"

services:

```
# -----
# PostgreSQL Database
# -----
db:
  image: postgres:16-alpine
  container_name: netconfig_db
  restart: unless-stopped
  environment:
    POSTGRES_DB: netconfig_diff_db
    POSTGRES_USER: netconfig_user
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-NetConfig@Secure2024!}
    PGDATA: /var/lib/postgresql/data/pgdata
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./backend/init_db.sql:/docker-entrypoint-initdb.d/init.sql:ro
  ports:
    - "5432:5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U netconfig_user -d netconfig_diff_db"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - netconfig_network
```

```
# -----
# Redis (Cache + Celery Broker)
# -----
```

```
redis:
  image: redis:7.4-alpine
  container_name: netconfig_redis
  restart: unless-stopped
  command: >
    redis-server
    --requirepass ${REDIS_PASSWORD:-RedisNetConfig2024!}
    --maxmemory 512mb
    --maxmemory-policy allkeys-lru
    --appendonly yes
  volumes:
    - redis_data:/data
  ports:
    - "6379:6379"
  healthcheck:
    test: ["CMD", "redis-cli", "--no-auth-warning", "-a",
      "${REDIS_PASSWORD:-RedisNetConfig2024!}", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - netconfig_network
```

```
# -----
# Ollama (Llama 3 LLM)
# -----

ollama:
  image: ollama/ollama:latest
  container_name: netconfig_ollama
  restart: unless-stopped
  ports:
    - "11434:11434"
  volumes:
    - ollama_data:/root/.ollama
    - ./scripts/pull_model.sh:/pull_model.sh:ro
  environment:
    OLLAMA_KEEP_ALIVE: "24h"
    OLLAMA_NUM_PARALLEL: "2"
    OLLAMA_MAX_LOADED_MODELS: "1"
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:11434/api/tags"]
    interval: 30s
    timeout: 15s
    retries: 5
    start_period: 60s
  networks:
    - netconfig_network

# -----
# Ollama Model Puller (init container pattern)
# -----

ollama-pull:
  image: ollama/ollama:latest
  container_name: netconfig_ollama_pull
  depends_on:
    ollama:
      condition: service_healthy
  entrypoint: >
    sh -c "ollama pull llama3 && echo 'Llama 3 model pulled successfully'"
  environment:
    OLLAMA_HOST: "http://ollama:11434"
  networks:
    - netconfig_network
  restart: "no"

# -----
# FastAPI Application
# -----

api:
  build:
    context: ./backend
    dockerfile: Dockerfile
    target: production
  container_name: netconfig_api
  restart: unless-stopped
  depends_on:
    db:
      condition: service_healthy
```

```
redis:
  condition: service_healthy
ollama:
  condition: service_healthy
environment:
  DATABASE_URL: "postgresql+asyncpg://netconfig_user:${POSTGRES_PASSWORD:-NetCon
  REDIS_URL: "redis://:${REDIS_PASSWORD:-RedisNetConfig2024!}@redis:6379/0"
  OLLAMA_BASE_URL: "http://ollama:11434"
  OLLAMA_MODEL: "llama3"
  SECRET_KEY: ${SECRET_KEY:-please-change-this-secret-key-in-production-minimum-
  ENVIRONMENT: ${ENVIRONMENT:-production}
  ALLOWED_ORIGINS: ${ALLOWED_ORIGINS:-http://localhost:3000}
  ALLOWED_HOSTS: "*"
volumes:
  - uploads_data:/tmp/uploads
  - reports_data:/tmp/reports
ports:
  - "8000:8000"
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8000/api/v1/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 20s
networks:
  - netconfig_network

# -----
# Celery Worker (Background AI Analysis Tasks)
# -----
celery-worker:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: netconfig_celery_worker
  restart: unless-stopped
  command: >
    celery -A app.workers.celery_app worker --
    loglevel=info
    --concurrency=4
    --pool=gevent
    -Q default,analysis,compliance,reports
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_healthy
    ollama:
      condition: service_healthy
  environment:
    DATABASE_URL: "postgresql+asyncpg://netconfig_user:${POSTGRES_PASSWORD:-NetCon
    REDIS_URL: "redis://:${REDIS_PASSWORD:-RedisNetConfig2024!}@redis:6379/0"
    OLLAMA_BASE_URL: "http://ollama:11434"
    OLLAMA_MODEL: "llama3"
    SECRET_KEY: ${SECRET_KEY:-please-change-this-secret-key-in-production}
```

volumes:

- uploads_data:/tmp/uploads -
- reports_data:/tmp/reports

networks:

- netconfig_network

Celery Beat (Scheduled Tasks)

celery-beat:

build:

context: ./backend

dockerfile: Dockerfile

container_name: netconfig_celery_beat

restart: unless-stopped

command: >

celery -A app.workers.celery_app beat

--loglevel=info

--scheduler=celery.beat.PersistentScheduler

depends_on:

- redis

environment:

REDIS_URL: "redis://:\${REDIS_PASSWORD:-RedisNetConfig2024!}@redis:6379/0"

networks:

- netconfig_network

Nginx Reverse Proxy

nginx:

image: nginx:1.27-alpine

container_name: netconfig_nginx

restart: unless-stopped

ports:

- "80:80"

- "443:443"

volumes:

- ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro

- ./nginx/ssl:/etc/nginx/ssl:ro

- reports_data:/var/www/reports:ro

depends_on:

api:

condition: service_healthy

networks:

- netconfig_network

Prometheus (Metrics)

prometheus:

image: prom/prometheus:v2.55.0

container_name: netconfig_prometheus

restart: unless-stopped

ports:

- "9090:9090"

```
volumes:
  - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
  - prometheus_data:/prometheus
```

```
command:
  - "--config.file=/etc/prometheus/prometheus.yml"
  - "--storage.tsdb.path=/prometheus"
  - "--storage.tsdb.retention.time=30d"
```

```
networks:
  - netconfig_network
```

```
# -----
# Grafana (Dashboards)
# -----
```

```
grafana:
  image: grafana/grafana:11.3.0
  container_name: netconfig_grafana
  restart: unless-stopped
  ports:
    - "3001:3000"
  volumes:
    - grafana_data:/var/lib/grafana
    - ./grafana/provisioning:/etc/grafana/provisioning:ro
  environment:
    GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD:-GrafanaNetConfig2024!}
    GF_USERS_ALLOW_SIGN_UP:"false"
    GF_SERVER_ROOT_URL:"http://localhost:3001"
  depends_on:
    - prometheus
  networks:
    - netconfig_network
```

```
# -----
# Database Migrations (init container)
# -----
```

```
db-migrate:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: netconfig_migrate
  command: alembic upgrade head
  depends_on:
    db:
      condition: service_healthy
  environment:
    DATABASE_URL: "postgresql+asyncpg://netconfig_user:${POSTGRES_PASSWORD:-NetCon
  networks:
    - netconfig_network
  restart: "no"
```

```
# -----
# Volumes
# -----
```

```
volumes:
  postgres_data:
    driver: local
```



```
redis_data:
  driver: local
ollama_data:
  driver: local
uploads_data:
  driver: local
reports_data:
  driver: local
prometheus_data:
  driver: local
grafana_data:
  driver: local
```

```
# -----
# Networks
# -----
networks:
  netconfig_network:
    driver: bridge
  ipam:
    config:
      - subnet: 172.20.0.0/16
```

6.3 — .env (Template)

```
bash

# Copy to .env and fill in values before docker-compose up
POSTGRES_PASSWORD=NetConfig@Secure2024!
REDIS_PASSWORD=RedisNetConfig2024!
SECRET_KEY=your-minimum-32-character-secret-key-here
ENVIRONMENT=production
GRAFANA_PASSWORD=GrafanaAdmin2024!
ALLOWED_ORIGINS=http://localhost:3000,http://your-android-app-ip
```

SECTION 7 — NGINX CONFIGURATION

```
nginx
```

```

# nginx/nginx.conf
events {
    worker_connections    1024;
    use epoll;
}

http {
    sendfile    on;
    tcp_nopush  on;
    keepalive_timeout    65;

    gzip    on;
    gzip_types application/json text/plain application/octet-stream;
    client_max_body_size    100M;

    # Rate limiting
    limit_req_zone    $binary_remote_addr zone=api_limit:10m rate=30r/s;
    limit_req_zone    $binary_remote_addr zone=upload_limit:10m rate=5r/s;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header Content-Security-Policy "default-src 'self'" always;
    server_tokens    off;

    upstream api_backend {
        server api:8000;
        keepalive    32;
    }

    server {
        listen    80;
        server_name _;

        location /api/ {
            limit_req zone=api_limit burst=50 nodelay;
            proxy_pass http://api_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_connect_timeout    30s;
            proxy_read_timeout    300s;
            proxy_send_timeout    30s;
        }

        location /api/v1/upload {
            limit_req zone=upload_limit burst=10 nodelay;
            client_max_body_size    100M;
            proxy_pass http://api_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_read_timeout    600s;
        }
    }
}

```

```
        location /metrics      {
            deny all;
        }

        location /health      {
            proxy_pass http://api_backend/api/v1/health;
        }
    }
}
```

SECTION 8 — ALEMBIC MIGRATIONS

Generate the initial Alembic migration file that creates all tables in correct dependency order:

1. `users`
2. `reviews` (FK → users)
3. `diff_changes` (FK → reviews)
4. `compliance_findings` (FK → reviews)
5. `audit_logs` (FK → users, reviews)
6. `workflow_steps` (FK → reviews, users)

Include proper indexes:

- `reviews.created_at` (BTREE, DESC)
- `reviews.status`
- `reviews.created_by`
- `diff_changes.review_id`
- `diff_changes.risk_level`
- `compliance_findings.review_id`
- `compliance_findings.status`
- `audit_logs.created_at` (BTREE, DESC)
- `audit_logs.user_id`
- `audit_logs.review_id`

SECTION 9 — ANDROID STUDIO CONFIGURATION NOTES

After generating the source code, perform these steps in Android Studio to build the APK:

Step 1: Configure BASE_URL for Ollama + Backend Connection

In `app/src/main/java/com/enterprise/netconfigdiff/Utils/Constants.kt`:

kotlin

```
object Constants {  
    // For emulator: use 10.0.2.2 (maps to host machine localhost)  
    // For real device on same WiFi: use your PC's LAN IP (e.g. 192.168.1.100)  
    const val BASE_URL = "http://10.0.2.2:80" // via Nginx proxy  
    const val OLLAMA_URL = "http://10.0.2.2:11434" // direct Ollama (for testing)  
    const val WEBSOCKET_URL = "ws://10.0.2.2:8000/ws"  
    const val DEFAULT_TIMEOUT_SECONDS = 30  
    const val UPLOAD_TIMEOUT_SECONDS = 300  
    const val ANALYSIS_POLL_INTERVAL_MS = 2000L  
    const val MAX_FILE_SIZE_MB = 50  
}
```

Step 2: network_security_config.xml

xml

```
<?xml version="1.0" encoding="utf-8"?>  
<network-security-config>  
    <domain-config cleartextTrafficPermitted="true">  
        <!-- Android emulator localhost -->  
        <domain includeSubdomains="true">10.0.2.2</domain>  
        <!-- LAN development machine -->  
        <domain includeSubdomains="true">192.168.1.0</domain>  
        <domain includeSubdomains="true">192.168.0.0</domain>  
    </domain-config>  
    <base-config cleartextTrafficPermitted="false">  
        <trust-anchors>  
            <certificates src="system"/>  
        </trust-anchors>  
    </base-config>  
</network-security-config>
```

Step 3: Signing Configuration (build.gradle.kts)

kotlin

```

android {
    signingConfigs {
        create("release") {
            storeFile = file("../keystore/netconfig-release.keystore")
            storePassword = System.getenv("KEYSTORE_PASSWORD") ?: "netconfig2024"
            keyAlias = System.getenv("KEY_ALIAS") ?: "netconfig"
            keyPassword = System.getenv("KEY_PASSWORD") ?: "netconfig2024"
        }
    }
    buildTypes {
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro")
            signingConfig = signingConfigs.getByName("release")
        }
        debug {
            applicationIdSuffix = ".debug"
            versionNameSuffix = "-DEBUG"
        }
    }
}

```

Step 4: Generate APK

In Android Studio:

1. Build → Generate Signed Bundle / APK
2. Select APK → Next
3. Create new keystore at `android/keystore/netconfig-release.keystore`
4. Fill Key Store Password, Key Alias, Key Password
5. Select `release` build variant → Finish
6. APK output: `android/app/build/outputs/apk/release/app-release.apk`

SECTION 10 — STARTUP INSTRUCTIONS

Start All Services (Docker Desktop)

```
bash
```

```
# 1. Copy and configure environment
cp .env.example .env
# Edit .env with your passwords

# 2. Start all containers
docker-compose up --build -d

# 3. Run database migrations
docker-compose exec api alembic upgrade head

# 4. Pull Llama 3 model (first time, ~5GB download)
docker exec -it netconfig_ollama ollama pull llama3

# 5. Verify all services are running
docker-compose ps

# 6. Check API health
curl http://localhost:80/api/v1/health

# 7. Access monitoring
# Grafana: http://localhost:3001 (admin / GrafanaNetConfig2024!)
# Prometheus: http://localhost:9090
# API Docs: http://localhost:8000/api/docs (dev only)
```

Stop All Services

```
bash

docker-compose down
# Preserve data volumes (recommended):
docker-compose down --volumes=false
# Nuclear option (removes all data):
docker-compose down -v
```

SECTION 11 — PRODUCTION CHECKLIST

Before deploying to production cloud environment, ensure:

- ☐ All passwords in `.env` changed from defaults
- ☐ `SECRET_KEY` is cryptographically random (minimum 64 chars)
- ☐ SSL certificates placed in `nginx/ssl/`
- ☐ `ENVIRONMENT=production` set (disables API docs)
- ☐ Database backups configured
- ☐ Firewall rules restrict Ollama (port 11434) to internal network only
- ☐ Grafana admin password changed
- ☐ Android APK signed with production keystore
- ☐ `BASE_URL` in Android app updated to production server IP/domain

END OF ANTIGRAVITY ENTERPRISE PROMPT

Generate every file listed in Section 2 with complete, production-grade, compilable code. No stubs. No TODO placeholders. No incomplete implementations. Every function must have a full body. Every class must have all properties and methods. Every Kotlin file must compile without errors under Android Studio Ladybug (2024.2.1) or later. The FastAPI backend must start cleanly with `uvicorn app.main:app` after `pip install -r requirements.txt`. The `docker-compose up --build` must succeed on Docker Desktop 4.34+ on Windows/Mac/Linux without manual intervention.